Project Phase 3 Report

Consultancy Appointment and Management System (CAMS)

Group 17 | Betül Merey 33963 Nehir Ceylan 35242

Database Description

CAMS is a relational database system designed to manage consultancy appointments between clients and professionals. It tracks services offered, locations used, payments made, and reviews submitted by clients. The schema consists of 7 tables: CLIENT, PROFESSIONAL, SERVICE, LOCATION, APPOINTMENT, PAYMENT (weak entity), and REVIEW (weak entity).

SQL Queries

Query 1 — List All Clients

Description: Retrieve all rows from the CLIENT table.
Category: A — Retrieve all rows

SQL:

```
SELECT * FROM CLIENT;
```

Relational Algebra:

```
CLIENT
```

Query 2 — Clients Born After 1990

Description: List the name, email, and date of birth of clients born after January 1, 1990.
Category: A — Selection + Projection

SQL:

```
SELECT name, email, date_of_birth
FROM CLIENT
WHERE date_of_birth > '1990-01-01';
```

Relational Algebra:

```
PI_name,email,date_of_birth (SIGMA_date_of_birth > '1990-01-01' (CLIENT))
```

Query 3 — Appointments with Client Names

Description: List each appointment ID and date along with the name of the client who booked it.
Category: B — Inner join between 2 tables

SQL:

```
SELECT A.appointment_id, A.date, C.name AS client_name
FROM APPOINTMENT A
JOIN CLIENT C ON A.client_id = C.client_id;
```

Relational Algebra:

```
PI_appointment_id,date,name (APPOINTMENT |X| CLIENT)
```

Query 4 — Total Amount Paid per Client

Description: Calculate the total payment amount for each client, sorted from highest to lowest.
Category: E — Join + Group By + Order By

SQL:

```
SELECT C.name AS client_name, SUM(P.amount) AS total_paid
FROM CLIENT C
JOIN APPOINTMENT A ON C.client_id = A.client_id
JOIN PAYMENT P ON A.appointment_id = P.appointment_id
GROUP BY C.name
ORDER BY total_paid DESC;
```

Relational Algebra:

```
GAMMA_C.name; SUM(amount)->total_paid (
  CLIENT |X| APPOINTMENT |X| PAYMENT
)
```

Query 5 — Professionals with More Than One Appointment

Description: List professionals who have conducted more than one appointment.
Category: E — Filtering grouped results with HAVING

SQL:

```
SELECT P.name, COUNT(A.appointment_id) AS appointment_count
FROM PROFESSIONAL P
JOIN APPOINTMENT A ON P.professional_id = A.professional_id
GROUP BY P.name
HAVING COUNT(A.appointment_id) > 1;
```

Relational Algebra:

```
SIGMA_count > 1 (
  GAMMA_name; COUNT(appointment_id)->count (
    PROFESSIONAL |X| APPOINTMENT
  )
)
```

Query 6 — All Services Ordered by Name

Description: List all services in ascending alphabetical order by service name.
Category: D — ORDER BY ascending

SQL:

```
SELECT service_id, service_name, description
FROM SERVICE
ORDER BY service_name ASC;
```

Relational Algebra:

```
SORT_service_name ASC (SERVICE)
```

Query 7 — Completed Payments Ordered by Amount

Description: List all completed payments sorted from highest to lowest amount.
Category: D — ORDER BY descending

SQL:

```
SELECT payment_id, appointment_id, amount, payment_method
FROM PAYMENT
```

```
WHERE payment_status = 'Completed'
ORDER BY amount DESC;
```

Relational Algebra:

```
SORT_amount DESC (SIGMA_payment_status='Completed' (PAYMENT))
```

Query 8 — Average Payment Amount per Method

Description: Calculate the average payment amount grouped by payment method.

Category: C — AVG with GROUP BY

SQL:

```
SELECT payment_method, AVG(amount) AS avg_amount
FROM PAYMENT
GROUP BY payment_method;
```

Relational Algebra:

```
GAMMA_payment_method; AVG(amount)->avg_amount (PAYMENT)
```

Query 9 — Appointment Details with Client, Professional, and Service

Description: List each appointment showing the client name, professional name, and service name.

Category: B — Join between 3 tables

SQL:

```
SELECT A.appointment_id, A.date,
       C.name AS client_name,
       P.name AS professional_name,
       S.service_name
FROM APPOINTMENT A
JOIN CLIENT       C ON A.client_id       = C.client_id
JOIN PROFESSIONAL P ON A.professional_id = P.professional_id
JOIN SERVICE      S ON A.service_id      = S.service_id;
```

Relational Algebra:

```
PI_appointment_id,date,C.name,P.name,service_name (
  APPOINTMENT |X| CLIENT |X| PROFESSIONAL |X| SERVICE
)
```

Query 10 — Clients with at Least One Completed Payment

Description: List clients who have made at least one payment with status 'Completed'.

Category: E — EXISTS

SQL:

```
SELECT C.name, C.email
FROM CLIENT C
WHERE EXISTS (
    SELECT 1
    FROM APPOINTMENT A
    JOIN PAYMENT P ON A.appointment_id = P.appointment_id
    WHERE A.client_id = C.client_id
      AND P.payment_status = 'Completed'
);
```

Relational Algebra:

```
PI_name,email (
  CLIENT |X| PI_client_id (
```

```
    SIGMA_payment_status='Completed' (APPOINTMENT |X| PAYMENT)
  )
)
```

Query 11 — Count of Appointments per Service

Description: Count how many appointments have been made for each service.

Category: C — COUNT with GROUP BY

SQL:

```
SELECT S.service_name, COUNT(A.appointment_id) AS total_appointments
FROM SERVICE S
JOIN APPOINTMENT A ON S.service_id = A.service_id
GROUP BY S.service_name;
```

Relational Algebra:

```
GAMMA_service_name; COUNT(appointment_id)->total (SERVICE |X| APPOINTMENT)
```

Query 12 — Maximum and Minimum Payment

Description: Show the highest and lowest payment amounts in the system.

Category: C — MAX and MIN

SQL:

```
SELECT MAX(amount) AS max_payment, MIN(amount) AS min_payment
FROM PAYMENT;
```

Relational Algebra:

```
GAMMA; MAX(amount)->max_payment, MIN(amount)->min_payment (PAYMENT)
```

Query 13 — Clients Not in Any Online Appointment

Description: List clients who have never had an appointment at an online location.

Category: E — NOT IN with subquery

SQL:

```
SELECT name, email
FROM CLIENT
WHERE client_id NOT IN (
    SELECT A.client_id
    FROM APPOINTMENT A
    JOIN LOCATION L ON A.location_id = L.location_id
    WHERE L.location_type = 'Online'
);
```

Relational Algebra:

```
PI_name,email (CLIENT) -
PI_name,email (CLIENT |X| APPOINTMENT |X|
  SIGMA_location_type='Online' (LOCATION)
)
```

Query 14 — Appointments Sorted by Date then Client

Description: List all appointments ordered by date ascending, then by client_id ascending.

Category: D — ORDER BY with multiple columns

SQL:

```
SELECT appointment_id, date, client_id, professional_id
FROM APPOINTMENT
ORDER BY date ASC, client_id ASC;
```

Relational Algebra:

```
SORT_date ASC, client_id ASC (APPOINTMENT)
```

Query 15 — Professionals with Average Rating Above 4

Description: List professionals whose appointments received an average review rating greater than 4.

Category: E — Join + Aggregate + HAVING

SQL:

```
SELECT P.name, AVG(R.rating) AS avg_rating
FROM PROFESSIONAL P
JOIN APPOINTMENT A ON P.professional_id = A.professional_id
JOIN REVIEW R ON A.appointment_id = R.appointment_id
GROUP BY P.name
HAVING AVG(R.rating) > 4;
```

Relational Algebra:

```
SIGMA_avg_rating > 4 (
  GAMMA_name; AVG(rating)->avg_rating (
    PROFESSIONAL |X| APPOINTMENT |X| REVIEW
  )
)
```

Appendix — AI Usage Documentation

AI Tool Used: Claude (Anthropic)

Prompt Used:

The Phase 1 ER diagram, Phase 2 relational schema, and Phase 3 assignment sheet were provided to Claude. Claude was asked to: (1) generate INSERT statements with at least 5 rows per table respecting all foreign key constraints, (2) write SQL queries covering all required categories (A through E), (3) write the corresponding Relational Algebra expression for each query, and (4) produce this Phase 3 PDF report.

AI Response Summary:

Claude generated all INSERT statements for 7 tables with realistic sample data. 15 queries were produced covering all required categories. Each query includes a description, SQL script, and Relational Algebra expression. The SQL file was structured to run without errors on the existing CAMS_DB schema from Phase 2.

Directly Used:

INSERT statements and SQL queries were used directly after review. The report structure and Relational Algebra expressions were also used as generated.

Modified:

Sample data values (names, emails, dates, amounts) were reviewed for realism and consistency with the CAMS domain.

Rejected:

No significant suggestions were rejected in this phase.